

Nome

Matr.

1. Per il seguente codice, ciascun record è istanziabile una o più volte e fa parte del sottosistema per il design pattern Facade:
 - (a) completare l'implementazione del pattern con la classe Facade Calcolatore che abbia i metodi appropriati,
 - (b) implementare una classe Client che ha a disposizione una configurazione di default di Calcolatore, chiede il costo totale, e aggiunge tipi di memoria, usando in modo appropriato le classi date e quella implementata nel punto a,
 - (c) disegnare il diagramma UML delle classi (indicando nomi classi, nomi metodi, e relazioni fra classi) per il codice fornito e quello aggiunto nei punti a, b,
 - (d) disegnare il diagramma UML di sequenza illustrando l'esecuzione a partire da una classe appropriata

```
record Memoria(int capienza, int velocita, int costo, String tipo) {  
    // tipo indica se cache, ram, ssd, disco  
}  
  
record SchedaGrafica(String risoluzione, int costo) {  
}  
  
record CPU(float frequenza, int numeroCore, String id, int costo) {  
}
```

2. Usando la programmazione funzionale di Java, data una lista tr di istanze di Triang, implementare una classe con:
 - (a) un metodo che restituisce la lista di istanze di TriangIso con i valori delle istanze della lista tr che sono triangoli isosceli, ovvero che due angoli siano uguali (nota: la base è il lato che ha misura diversa dagli altri due lati, il vertice è l'angolo che ha la misura diversa dagli altri due angoli),
 - (b) un metodo che a partire dalla lista di TriangIso trovata restituisce una lista contenente la base dei triangoli isosceli,
 - (c) un metodo che restituisce il vertice di misura maggiore fra i triangoli isosceli,
 - (d) un metodo che restituisce la somma dei perimetri delle istanze di tr.

```
record Triang(int lato1, int lato2, int lato3, int angolo1, int angolo2, int angolo3) {  
}  
  
record TriangIso(int lato, int base, int vertice) {  
}
```